

GNS3 Python

Network Automation Suite

Complete Project Guide — Setup, Code & Commands

Device	Host	Telnet Port	Role
R2	192.168.80.129	5003	Router 2
R3	192.168.80.129	5002	Router 3
Cloud1	192.168.80.129	—	NAT Bridge
GNS3 VM	192.168.80.129	80	REST API Server

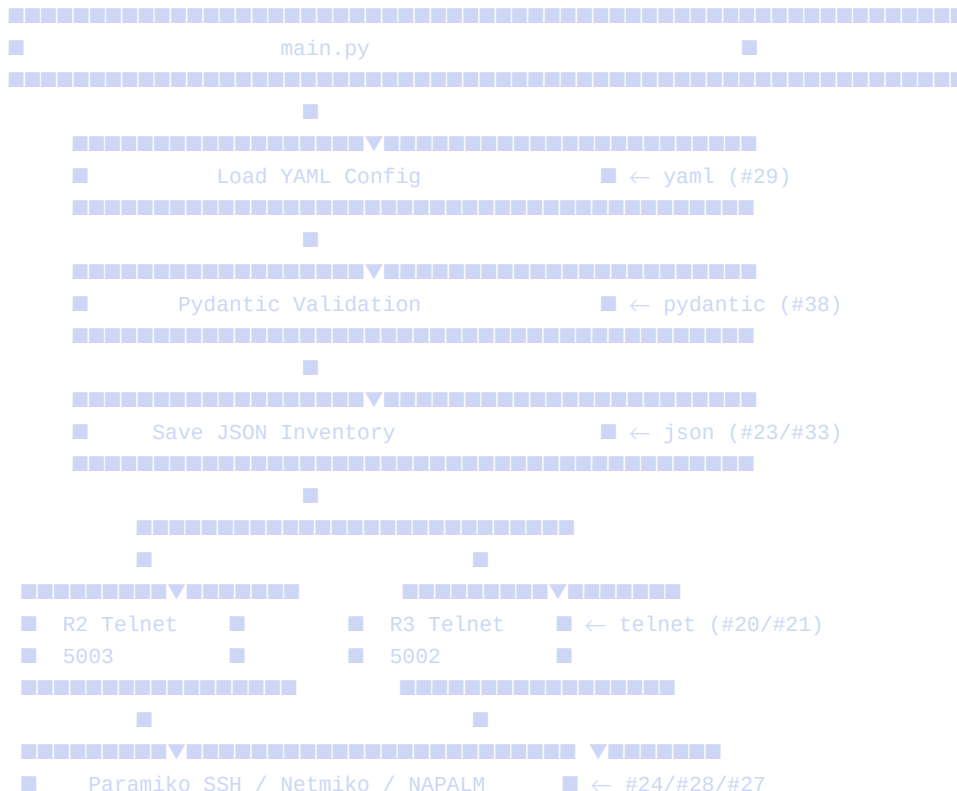
38 Python Topics Covered

Strings	For-loops	Numbers	Operators	Variables	While Loop	Dictionary	Lists	Tuples	Sets
Functions	Conditional	Exception Handling	Adv. Functions	Lambda	List Comprehension	SSH Config	Multithreading	Regex	Telnet
Telnet Library	POST APIs	JSON API	SSH Paramiko	PyClasses	PyModule	NAPALM	Netmiko	YAML	DHCP Config
Paramiko (adv)	Device Automation		Config Gen		Netconf/YANG		Send Commands		Multiline Cr

Project Structure & Architecture

network_automation_project/	
■■■ main.py	← Entry point – runs all 11 phases
■■■ requirements.txt	← pip dependencies
■■■ README.md	
■■■ config/	
■ ■■■ devices.yaml	← YAML device inventory (#29)
■■■ inventory/	
■ ■■■ devices.json	← JSON inventory store (#23/#33)
■■■ logs/	
■ ■■■ automation.log	← Auto-generated logs (#31)
■■■ reports/	
■ ■■■ report_YYYYMMDD.txt	← Automation report (#32)
■ ■■■ R2_config.txt	← Generated R2 config (#34)
■ ■■■ R3_config.txt	← Generated R3 config (#34)
■■■ modules/	← Python package (#26)
■■■ __init__.py	
■■■ device_model.py	← Pydantic models (#38)
■■■ inventory_manager.py	← JSON CRUD (#23/#7/#8)
■■■ telnet_module.py	← Telnet automation (#20/#21)
■■■ ssh_paramiko.py	← Paramiko SSH (#24/#17)
■■■ netmiko_module.py	← Netmiko suite (#28/#32/#36/#37)
■■■ napalm_module.py	← NAPALM getters (#27)
■■■ api_module.py	← GNS3 REST API (#22/#23)
■■■ sniff_module.py	← Scapy + Threads (#28/#18)
■■■ log_parser.py	← Log parsing + Regex (#31/#19)
■■■ config_gen.py	← Config generation (#34)
■■■ report_gen.py	← Report generation (#32)

Data Flow Diagram



Prerequisites & Installation Guide

Step 1 — GNS3 Lab Topology (Already Done ✓)

Your GNS3 topology is already set up as seen in the screenshots. Confirm the following before running the Python project:

✓	GNS3 VM running in VMware at 192.168.80.129
✓	R2 connected via Telnet at port 5003
✓	R3 connected via Telnet at port 5002
✓	Cloud1 bridging R2 and R3
✓	GNS3 Web UI: http://192.168.80.129 (admin/admin)
✓	Cisco IOS image loaded (c3725 or similar)

Step 2 — Install Python & VS Code

Python 3.10+ is required. Download from python.org if not installed.

```
$ python --version
```

■ Should show Python 3.10.x or higher

In VS Code, install these extensions:

- Python (Microsoft)
- Pylance
- YAML (Red Hat)

Step 3 — Create Project Folder & Virtual Environment

```
$ mkdir network_automation_project && cd network_automation_project
```

```
$ python -m venv venv
```

```
$ venv\Scripts\activate # Windows
```

```
$ source venv/bin/activate # Linux/Mac
```

■ Always activate venv before running the project

Step 4 — Install All Dependencies

```
$ pip install netmiko napalm paramiko pydantic pyyaml scapy requests
```

Package	Version	Purpose	Topics
netmiko	>=4.3.0	Multi-vendor SSH automation	#28,#32,#36,#37
napalm	>=4.1.0	Network device abstraction	#27
paramiko	>=3.3.1	Low-level SSH client	#24,#17
pydantic	>=2.5.0	Data validation models	#38
pyyaml	>=6.0.1	YAML file parsing	#29

scapy	>=2.5.0	Packet capture/crafting	#28,#18
requests	>=2.31.0	HTTP REST API calls	#22,#23

Router Initial Setup (Manual via GNS3 Console)

Before running the Python automation, paste these commands manually into each router's GNS3 console. This creates the initial user account and enables Telnet access so the Python scripts can connect.

R2 — Initial Console Commands

```
! ■■ Open GNS3 Console for R2 and paste:
Router> enable
Router# conf t
Router(config)# hostname R2
R2(config)# no ip domain-lookup
R2(config)# username cisco privilege 15 secret cisco
R2(config)# enable secret cisco
R2(config)# interface FastEthernet0/0
R2(config-if)# ip address 10.0.0.1 255.255.255.252
R2(config-if)# no shutdown
R2(config-if)# exit
R2(config)# line vty 0 4
R2(config-line)# login local
R2(config-line)# transport input telnet ssh
R2(config-line)# exit
R2(config)# line con 0
R2(config-line)# logging synchronous
R2(config-line)# exit
R2(config)# end
R2# write memory
```

R3 — Initial Console Commands

```
! ■■ Open GNS3 Console for R3 and paste:
Router> enable
Router# conf t
Router(config)# hostname R3
R3(config)# no ip domain-lookup
R3(config)# username cisco privilege 15 secret cisco
R3(config)# enable secret cisco
R3(config)# interface FastEthernet0/0
R3(config-if)# ip address 10.0.0.2 255.255.255.252
R3(config-if)# no shutdown
R3(config-if)# exit
R3(config)# line vty 0 4
R3(config-line)# login local
R3(config-line)# transport input telnet ssh
R3(config-line)# exit
R3(config)# line con 0
R3(config-line)# logging synchronous
R3(config-line)# exit
R3(config)# end
R3# write memory
```

Verify Connectivity Between R2 and R3

■ From R2 console — ping R3's IP:

```
R2# ping 10.0.0.2
!!!!!      ← 5 exclamation marks = full connectivity
```

■ From R3 console — ping R2's IP:

```
R3# ping 10.0.0.1
!!!!!
```

Key Code Sections — All 38 Topics

Section 1 — Core Python Fundamentals (#1–#10)

Strings (#1), Variables (#5), Numbers (#3), Operators (#4)

```
# In device_model.py – Pydantic validation + string formatting
name      = "R2"                                # String variable
ip        = "10.0.0.1"
port      = 5003                                # Number
timeout   = port * 2                            # Operator: multiplication
summary   = f"[{name}] host={ip}:{port}" # f-string formatting

# String methods
if name.strip().upper() == "R2":                # .strip(), .upper()
    prefix = "ROUTER_" + name                  # String concatenation
```

For-loops (#2), Lists (#8), List Comprehension (#16)

```
# In netmiko_module.py
commands = ["show version", "show ip int brief", "show bgp summary"]

# For-loop over commands
for cmd in commands:
    output = conn.send_command(cmd)
    print(f" {cmd}: {len(output)} chars")

# List comprehension – filter only 'up' interfaces
up_interfaces = [
    iface["interface"] for iface in interfaces
    if iface["status"] == "up"
]
```

While Loop (#6), Conditional (#12)

```
# In telnet_module.py – retry prompt detection
attempts = 0
while attempts < 3:                                # While loop
    probe = telnet_send(tn, "")
    if "#" in probe or ">" in probe:                # Conditional
        break                                       # Exit while loop
    attempts += 1
    time.sleep(0.5)
```

Dictionary (#7), Tuples (#9), Sets (#10)

```
# Dictionary – device info
device = {
    "name": "R2", "host": "192.168.80.129",
    "telnet_port": 5003, "platform": "cisco_ios"
}

# Tuple – immutable host/port pair
host, port = device_model.to_telnet_tuple() # ("192.168.80.129", 5003)

# Set – unique platforms (no duplicates)
platforms = {d["platform"] for d in inventory}
# → {'cisco_ios'}
```


Functions, Lambda & Advanced Features

Functions (#11), Exception Handling (#13)

```
# In inventory_manager.py
def load_inventory() -> list:
    """Load JSON inventory with exception handling."""
    try:
        with open("inventory/devices.json", "r") as f:
            data = json.load(f)
        return data
    except FileNotFoundError:          # Specific exception
        print("No inventory file found")
        return []
    except json.JSONDecodeError as e:  # Catch JSON errors
        print(f"JSON error: {e}")
        return []
```

Advanced Functions (#14) — Decorator with retry

```
# In ssh_paramiko.py
def retry(max_attempts=3, delay=2.0):
    """Decorator factory – retry a function on failure."""
    def decorator(func):
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f"Attempt {attempt} failed: {e}")
                    if attempt < max_attempts:
                        time.sleep(delay)
            raise RuntimeError(f"All {max_attempts} attempts failed")
        return wrapper
    return decorator

@retry(max_attempts=3, delay=2.0)  # Apply decorator
def connect(self):
    self.client.connect(...)
```

Lambda Functions (#15)

```
# In ssh_paramiko.py + log_parser.py

# Lambda: clean whitespace from a line
clean = lambda line: line.strip()
lines = list(map(clean, raw_lines))          # map() with lambda

# Lambda: filter blank lines
non_empty = list(filter(lambda l: len(l) > 5, lines))

# Lambda: combine config sections
combine = lambda *parts: "\n!\n".join(parts)
full_config = combine(base, bgp, acl, dhcp)
```

Networking Modules — Telnet, SSH, Netmiko, NAPALM

Telnet Library (#20, #21)

```
# In telnet_module.py
import telnetlib                                # Telnet Library (#21)

tn = telnetlib.Telnet("192.168.80.129", 5003, timeout=10)
tn.read_until(b"Username:", timeout=10)
tn.write(b"cisco\n")
tn.read_until(b"Password:", timeout=10)
tn.write(b"cisco\n")
tn.write(b"enable\n")
tn.write(b"cisco\n")

# Send a config command
tn.write(b"conf t\n")
tn.write(b"hostname R2\n")
tn.write(b"end\n")
output = tn.read_very_eager().decode("utf-8")
tn.close()
```

SSH Configuration (#17) — Enable SSH on Router

```
# Via Telnet – configure SSH on the router
ssh_commands = [
    "conf t",
    "ip domain-name lab.local",
    "crypto key generate rsa modulus 1024",
    "ip ssh version 2",
    "username cisco privilege 15 secret cisco",
    "line vty 0 4",
    "transport input ssh telnet",
    "login local",
    "exit",
    "end",
    "write memory",
]
for cmd in ssh_commands:
    telnet_send(tn, cmd, wait=1.5)
```

SSH Paramiko (#24)

```
# In ssh_paramiko.py
import paramiko

client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect(
    hostname="192.168.80.129",
    port=22,
    username="cisco",
    password="cisco",
    look_for_keys=False
)
shell = client.invoke_shell()
time.sleep(1)
```


Netmiko — Complete Feature Set (#28, #32, #36, #37)

Netmiko Basic Connection & Send Commands (#36)

```
# In netmiko_module.py
from netmiko import ConnectHandler

device = {
    "device_type": "cisco_ios",
    "host":        "192.168.80.129",
    "port":        22,
    "username":    "cisco",
    "password":    "cisco",
    "secret":      "cisco",
}

conn = ConnectHandler(**device)
conn.enable()

# Send single show command (#36)
output = conn.send_command("show ip interface brief")
print(output)
conn.disconnect()
```

Netmiko Multiline Config Push (#37)

```
# Send a block of config commands (#37)
config_commands = [
    "interface Loopback0",
    "ip address 2.2.2.2 255.255.255.255",
    "no shutdown",
    "description LOOPBACK_R2",
]

output = conn.send_config_set(config_commands) # Multiline (#37)
conn.save_config()                           # write memory
print(output)
```

Netmiko — BGP Configuration (#32)

```
bgp_commands = [
    "router bgp 200",
    "bgp router-id 2.2.2.2",
    "neighbor 10.0.0.2 remote-as 300",
    "address-family ipv4",
    "    network 10.0.0.0 mask 255.255.255.252",
    "    network 2.2.2.2 mask 255.255.255.255",
    "    neighbor 10.0.0.2 activate",
    "exit-address-family",
]

output = conn.send_config_set(bgp_commands)
```

Apply ACL (#32)

```
acl_commands = [
    "ip access-list extended BLOCK_TELNET",
```


NAPALM, GNS3 API, YAML, Pydantic

NAPALM (#27)

```
from napalm import get_network_driver

driver = get_network_driver("cisco_ios")

optional_args = {"transport": "telnet", "port": 5003}

with driver("192.168.80.129", "cisco", "cisco",
            optional_args=optional_args) as device:
    facts      = device.get_facts()
    interfaces = device.get_interfaces()
    bgp        = device.get_bgp_neighbors()

print(facts["hostname"])    # → R2
print(facts["os_version"])  # → 12.4 / 15.x
```

Python YAML (#29)

```
import yaml

# Load YAML config file
with open("config/devices.yaml", "r") as f:
    config = yaml.safe_load(f)

# Access nested YAML data
devices = config["devices"]          # list of dicts
for dev in devices:
    print(dev["name"], dev["host"], dev["telnet_port"])
# → R2  192.168.80.129  5003
# → R3  192.168.80.129  5002
```

GNS3 REST API — POST & JSON (#22, #23)

```
import requests, json

BASE = "http://192.168.80.129/v2"
AUTH = ("admin", "admin")

# GET all projects
r = requests.get(f"{BASE}/projects", auth=AUTH)
projects = r.json()          # JSON API (#23)

# POST to start all nodes
project_id = projects[0]["project_id"]
r = requests.post(           # POST API (#22)
    f"{BASE}/projects/{project_id}/nodes/start",
    json={},                 # JSON body
    auth=AUTH
)
print(r.status_code)        # → 204
```

Pydantic Models (#38)

```
from pydantic import BaseModel, field_validator
```


Regex, Log Parsing, Multithreading, Packet Sniff

Regex (#19)

```
import re

# Extract interface status from show interfaces output
pattern = re.compile(
    r"^(\S+) is (up|down),\s+line protocol is (up|down)",
    re.MULTILINE
)
matches = pattern.findall(raw_output)
# → [("FastEthernet0/0", "up", "up")]

# Extract all IPs from text
ip_pattern = re.compile(r"\b(?:\d{1,3}\.){3}\d{1,3}\b")
ips = ip_pattern.findall(text)
# → ["10.0.0.1", "10.0.0.2", "2.2.2.2"]

# Find BGP syslog events
bgp_up = re.compile(r"%BGP-5-ADJCHANGE:\s+neighbor\s+(\S+)\s+Up")
for line in log_lines:
    m = bgp_up.search(line)
    if m:
        print(f"BGP neighbor {m.group(1)} came up!")
```

Log Parsing (#31)

```
# Write to log
def write_log(message, level="INFO"):
    ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open("logs/automation.log", "a") as f:
        f.write(f"[{ts}] [{level}] {message}\n")

# Parse log for events
events = parse_log_file("logs/automation.log")
summary = get_log_summary(events)
# summary → {"bgp_neighbor_up": 2, "interface_up": 3, ...}
```

Multithreading (#18)

```
import threading

results = {}
lock = threading.Lock()

def worker(device):
    out = netmiko_configure_loopback(device)
    with lock:
        # Thread-safe write
        results[device["name"]] = out

# Create one thread per device
threads = []
for device in devices:
    t = threading.Thread(target=worker, args=(device,))
    threads.append(t)
    t.start()
```



```
for t in threads:    # Wait for all to finish
    t.join()
```

Packet Sniffing (#28) — Scapy

```
from scapy.all import sniff, IP, TCP, ICMP

def packet_callback(packet):
    if IP in packet:
        print(f"{packet[IP].src} → {packet[IP].dst}")
    if TCP in packet:
        print(f"  TCP port {packet[TCP].dport}")

# Sniff 20 packets on eth0, 10 second timeout
sniff(iface="eth0", prn=packet_callback, count=20, timeout=10)
```

DHCP, Config Generation, Report & Run Guide

DHCP Pool Configuration (#30)

Config Generation (#34)

Running the Complete Project

After completing all setup steps above, run the project from VS Code terminal:

```
$ cd network_automation_project
```

```
$ python main.py
```

- The project runs 11 phases sequentially. Each phase is independent — if SSH/Telnet fails (device not reachable), it logs the error and continues.

Expected Output

```

Expected Output
■ GNS3 PYTHON NETWORK AUTOMATION SUITE ■
■
[YAML] Loaded config: config/devices.yaml
[STEP 1] Validating device models...

```


Complete Topic Index — All 38 Topics

#	Topic	Module File	Key Code Element
1	Strings	device_model.py	f-strings, .strip(), .upper()
2	For-loops	netmiko_module.py	for cmd in commands
3	Numbers	telnet_module.py	port, timeout, counts
4	Operators	inventory_manager.py	+=, <=, ==, not
5	Variables	all modules	name, ip, port, mask
6	While Loop	telnet_module.py	while attempts < 3
7	Dictionary	inventory_manager.py	device = {name: ..}
8	Lists	netmiko_module.py	commands = [...]
9	Tuples	device_model.py	to_telnet_tuple()
10	Sets	inventory_manager.py	{d['platform'] for d}
11	Functions	all modules	def load_inventory()
12	Conditional	log_parser.py	if state == 'up'
13	Exception Handling	telnet_module.py	try/except/finally
14	Adv. Functions	ssh_paramiko.py	@retry decorator
15	Lambda	ssh_paramiko.py	clean = lambda l: l.strip()
16	List Comprehension	netmiko_module.py	[x for x in y if ...]
17	SSH Configuration	telnet_module.py	crypto key generate rsa
18	Multithreading	sniff_module.py	threading.Thread(...)
19	Regex	log_parser.py	re.compile(r'...')
20	Telnet Script	telnet_module.py	telnet_configure_basic()
21	Telnet Library	telnet_module.py	import telnetlib
22	POST APIs	api_module.py	requests.post(url, json={})
23	JSON API	api_module.py	json.load() / resp.json()
24	SSH Paramiko	ssh_paramiko.py	paramiko.SSHClient()
25	PyClasses	ssh_paramiko.py	class ParamikoSSH
26	PyModule Package	modules/_init_.py	__all__ = [...]
27	NAPALM	napalm_module.py	get_network_driver()
28	Netmiko/Packet Sniff	netmiko_module.py	ConnectHandler()
29	Python YAML	main.py	yaml.safe_load(f)
30	DHCP Pool Config	netmiko_module.py	netmiko_configure_dhcp()
31	Log Parsing	log_parser.py	parse_log_file()
32	Netmiko (advanced)	netmiko_module.py	BGP/ACL/Loopback/Report
33	Device Automation	inventory_manager.py	build_inventory_from_yaml()
34	Config Generation	config_gen.py	generate_all_configs()
35	Netconf/YANG	netmiko_module.py	send_config_set (IOS-XE)
36	Send Commands	netmiko_module.py	conn.send_command()
37	Multiline Commands	netmiko_module.py	conn.send_config_set([])

38	Pydantic	device_model.py	class DeviceModel(BaseModel)
----	----------	-----------------	------------------------------